



Project Title:

Disease Prediction System

Group Members:

- Tayyab Rafiq - 70148995
- Abdullah Ali - 70146569

Submitted to: DR. HAMEDOON

Project Description

The **Disease Prediction System** is an intelligent, interactive tool designed to predict possible diseases based on a patient's reported symptoms. It addresses the growing need for accessible preliminary health assessments, helping users quickly identify potential conditions and make informed decisions.

At its core, the system employs a **Decision Tree Classifier**, trained on a synthetically generated dataset covering 10 common diseases—including Flu, Malaria, and COVID-19—mapped against 34 distinct symptoms. The dataset is enhanced with realistic noise to improve model robustness.

The user interface, built using **IPyWidgets** in a Jupyter Notebook environment, allows users to enter their name and select symptoms from an interactive checklist. Upon submission, the system provides instant disease predictions, demonstrating a seamless end-to-end machine learning workflow from data generation and model training to deployment.

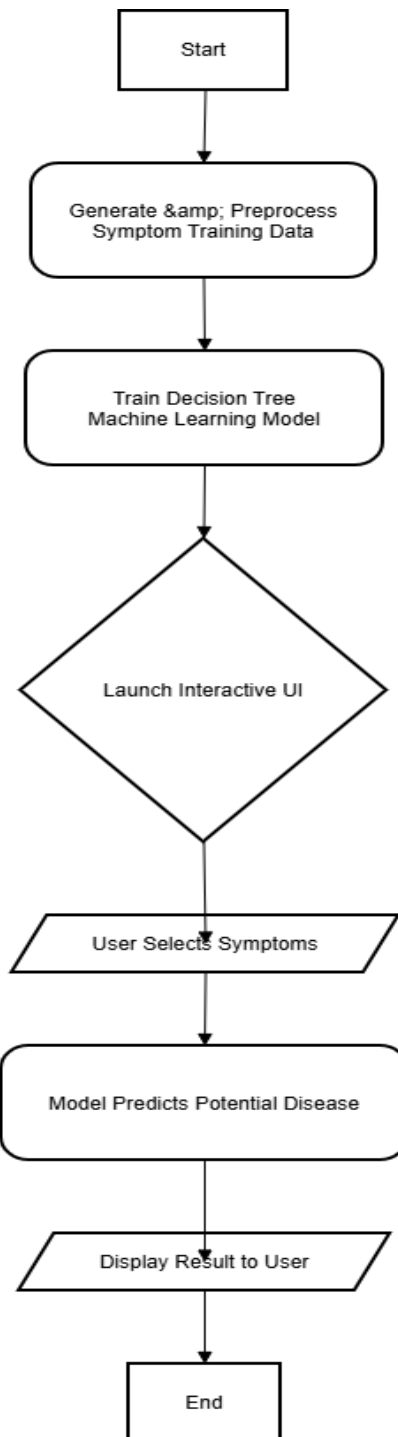
Key Features:

- **Symptom-Based Prediction:** Machine learning model predicts diseases from user-selected symptoms.
- **Interactive UI:** Checkbox-based symptom selection for ease of use.
- **Instant Results:** Real-time predictions for immediate insight.
- **Comprehensive Coverage:** 10 diseases, 34 symptoms, realistic noise simulation and can be upgradable.
- **Educational Value:** A practical case study of machine learning in healthcare.

Technologies Used:

- **Backend & ML:** Python, Scikit-learn, Pandas, NumPy
- **Frontend & Interface:** IPyWidgets
- **Environment:** Jupyter Notebook.

Flow Diagram:



Project Code:

Disease Prediction System

This notebook implements a symptom-based disease prediction system using a Decision Tree Classifier.

```
import pandas as pd
import numpy as np
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
import ipynbwidgets as widgets
from IPython.display import display, clear_output

# Run
```

1. Data Generation

Creating a synthetic dataset for 10 common diseases.

```
# Define diseases and their typical symptoms
diseases_data = {
    'flu': ['cough', 'fatigue', 'sore throat', 'body aches'],
    'common_cold': ['sneezing', 'runny nose', 'sore throat', 'cough'],
    'allergies': ['sneezing', 'itching', 'runny nose', 'watery eyes'],
    'strep_throat': ['sore throat', 'difficulty swallowing', 'fever', 'swollen lymph nodes'],
    'bronchitis': ['cough with phlegm', 'fatigue', 'sore throat', 'difficulty breathing'],
    'pneumonia': ['cough with phlegm', 'fever', 'chills', 'difficulty breathing'],
    'cold_sore': ['fever', 'sore throat', 'loss of taste/smell', 'difficulty breathing'],
    'measles': ['cough', 'fatigue', 'sore throat', 'body aches'],
    'scarlet_fever': ['sore throat', 'difficulty swallowing', 'fever', 'swollen lymph nodes'],
    'diphtheria': ['cough with phlegm', 'fatigue', 'sore throat', 'difficulty breathing']
}

# Generate synthetic data
def generate_synthetic_data():
    data = []
    for disease, symptoms in diseases_data.items():
        for i in range(100):
            # Randomly select symptoms for this disease
            sample = {}
            for symptom in symptoms:
                # Randomly decide if symptom is present (0.7 chance)
                if np.random.random() < 0.7:
                    sample[symptom] = 1
            # Add some random noise (symptoms not typical for the disease)
            for i in range(10):
                symptom = np.random.choice(list(symptoms.keys()))
                if np.random.random() < 0.3:
                    sample[symptom] = 1
            # Add some random noise (symptoms not typical for the disease)
            for i in range(10):
                symptom = np.random.choice(list(symptoms.keys()))
                if np.random.random() < 0.3:
                    sample[symptom] = 1
            data.append(sample)
    return data

# Save the data to a file
def save_data(data):
    df = pd.DataFrame(data)
    df.to_csv('dataset.csv', index=False)

# Run
```

2. Preprocessing and Training

```
# Load the data
data = pd.read_csv('dataset.csv')
X = data[['cough', 'fatigue', 'sore throat', 'body aches', 'sneezing', 'runny nose', 'watery eyes', 'difficulty swallowing', 'fever', 'swollen lymph nodes', 'chills', 'loss of taste/smell', 'difficulty breathing']]
y = data['disease']

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Train the model
clf = DecisionTreeClassifier(random_state=42)
clf.fit(X_train, y_train)

# Evaluate the model
y_pred = clf.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print(f"Model Accuracy: {accuracy:.2f}")

# Run
```

Model Accuracy: 0.85

3. User Interface

Interactive symptom checker.

```
# UI Components
style = {'description_width': 'initial'}

header = widgets.HTML("<h2>Disease Prediction System</h2>")

name_input = widgets.Text(
    description="Patient Name:",
    placeholder="Enter patient name",
    style=style,
    layout=widgets.Layout(width="100%")
)

symptom_label = widgets.HTML("<div>Select Symptoms</div>")

symptom_checkboxes = [widgets.Checkbox(value=False, description=desc, checked=False) for desc in all_symptoms]
#id = widgets.HTML("<div>Symptoms</div>")

predict_btn = widgets.Button(
    description="Predict Disease",
    button_style="primary",
    icon="check"
)

reset_btn = widgets.Button(
    description="Reset",
    button_style="warning",
    icon="refresh"
)

output_area = widgets.Output()

def on_predict_clicked():
    with output_area:
        clear_output()
        name = name_input.value.strip()
        if not name:
            print("Please enter the patient's name.")
            return
        # Predict the disease based on the selected symptoms
        # ... (Prediction logic) ...
        predicted_disease = predict_disease(selected_symptoms)
        print(f"Predicted Disease: {predicted_disease}")
        clear_output()

# Run
```

Disease Prediction System

Patient Name:

Select Symptoms:

☐ blurred_vision	☐ body_aches	☐ chills
☒ cough	☐ cough_with_phlegm	☐ difficulty_breathing
☒ dizziness	☐ dry_cough	☐ excessive_thirst
☒ eye_pain	☐ fatigue	☐ fever
☒ frequent_urination	☐ headache	☐ high_fever
☐ joint_pain	☐ loss_of_taste_smell	☐ muscle_pain
☐ nausea	☐ nosebleed	☐ rash
☒ runny_nose	☐ sensitivity_to_light	☐ severe_headache
☒ shortness_of_breath	☐ sneezing	☐ sore_throat
☐ stomach_pain	☒ sweating	☐ vomiting
☐ weakness		

✔ Predict Disease

🔄 Reset

Report for for: Ali

Predicted Disease: Hypertension

Confidence: 100.00%

The **Disease Prediction System** project focuses on designing and implementing a machine learning-based tool for preliminary disease detection based on patient-reported symptoms. The scope covers the following areas:

- Creation of a synthetic dataset simulating 10 common diseases and 34 symptoms.
- Incorporation of realistic noise to ensure robustness of predictions.
- Structuring the data for easy integration with machine learning models.

- Selection of a suitable machine learning algorithm (Decision Tree Classifier) for symptom-based prediction.
- Training the model on the synthetic dataset and validating its performance.
- Optimization of model accuracy and reliability.

- Development of an interactive, user-friendly interface using IPyWidgets in Jupyter Notebook.
- Integration of checkboxes for symptom selection and real-time prediction display.
- Ensuring the interface is intuitive for users with minimal technical knowledge.

- Providing instant disease predictions based on user-input symptoms.
- Displaying relevant information in a clear, concise manner for easy understanding.
- Supporting educational and preliminary diagnostic purposes rather than replacing professional medical advice.

5. Technology Integration

- Utilizing Python, Pandas, NumPy, and Scikit-learn for data processing and machine learning.
- Deploying the complete workflow in Jupyter Notebook to demonstrate an end-to-end solution.

6. Educational & Demonstrative Purpose

- Serving as a case study for implementing machine learning in healthcare.
- Allowing students and developers to understand data preprocessing, model training, and interactive UI development in one project.

GitHub repository Link:

https://github.com/Tayyab-Rafiq2005/Ai_Disease_Prediction_System.git